



Project name: University system

Discipline: Object-Oriented programming

Team members:

Gorbachyov Artyom

Sakenuly Daniyar

Knyazev Artyom

1. Introduction

UniCRM is a university management system developed in Java using Object-Oriented Programming principles. The system is designed using layered architecture to separate responsibilities between business logic, data persistence, commands, and domain entities. The project supports academic management, communication, technical support, and research-related functionality.

2. Project Goals

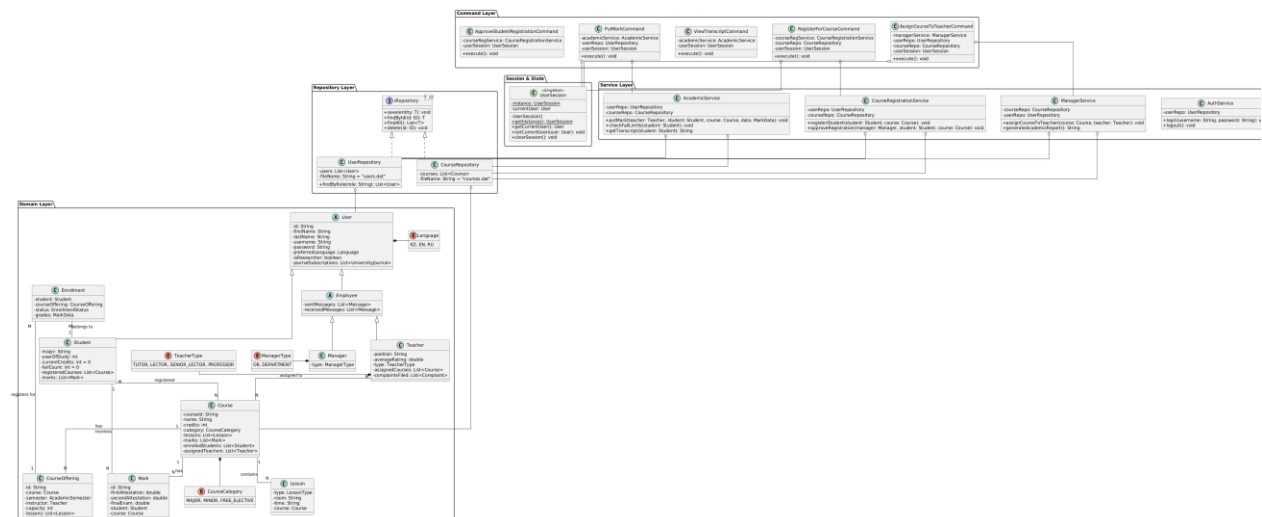
The main goal of the project was to develop a console-based university intranet system that simplifies interaction between students, teachers, managers, and technical support specialists. The project demonstrates practical implementation of OOP principles, UML modeling, design patterns, and JSON-based persistence.

3. Project Architecture

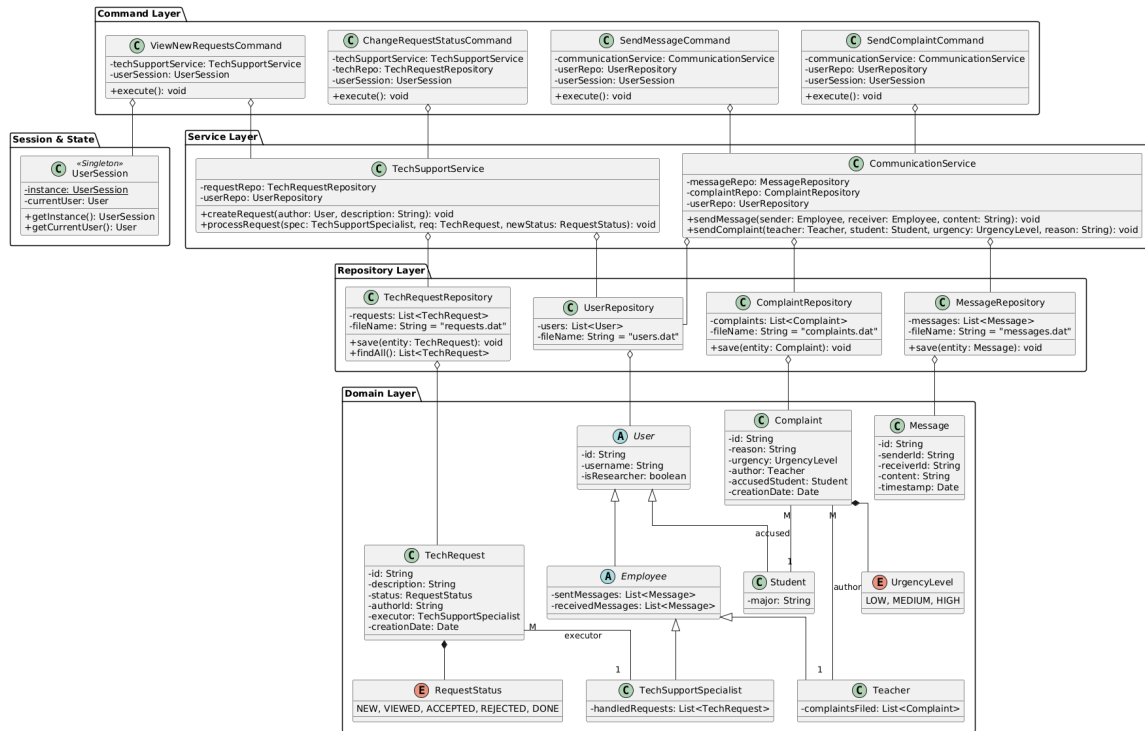
The project uses layered architecture. Each layer is responsible for a separate part of the system.

- **Domain Layer:** Contains main entities such as User, Student, Teacher, Course, Enrollment, ResearchPaper, ResearchProject, and UniversityJournal.
- **Repository Layer:** Responsible for saving and loading data using JSON files such as users.json, courses.json, journals.json, and research_projects.json.
- **Service Layer:** Contains business logic including registration, grading, publication of research papers, citation generation, and technical support management.
- **Command Layer:** Implements the Command pattern where every user action is represented by a separate command class.
- **Session & State:** Handles current user session using the Singleton pattern through UserSession.

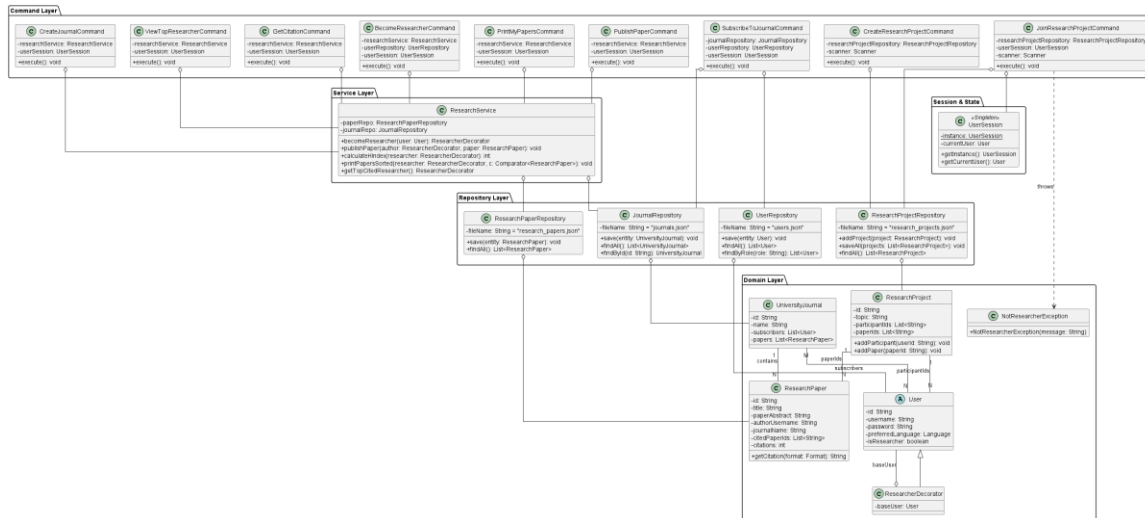
4. UML Diagrams



COMMUNICATION CLASS DIAGRAM



RESEARCH CLASS DIAGRAM



5. Project Structure

The project is divided into several packages:

- command – contains command classes for menu actions
- domain – contains system entities and domain models
- repository – responsible for JSON persistence
- service – contains business logic

- session – manages active user session
- localization – responsible for multilingual interface
- exception – custom exceptions

6. Academic Management Module

This module handles course registration, enrollments, schedules, grading, and transcripts.

Workflow example:

1. Manager creates course
2. Manager creates course offering
3. Teacher is assigned to course offering
4. Student registers for offering
5. Manager approves enrollment
6. Teacher puts marks
7. Transcript is updated

CreateCourseOfferingCommand

```
// Create offering and generate schedule
CourseOffering offering = offeringService.createOffering(course, semester, teacher, capacity);
boolean scheduled = scheduleService.generateSchedule(offering);

if (!scheduled) {
    System.out.println(localization.get(LocalizationKey.NO_ROOMS_FOUND));
}

offeringService.save(offering);
System.out.println(localization.get(LocalizationKey.OFFERING_CREATED));
```

RegisterForOfferingCommand

```
CourseOffering offering = offerings.get(idx);
enrollmentService.registerStudent(student, offering);
System.out.println(localization.get(LocalizationKey.ENROLLMENT_CREATED_PENDING));
```

```
System.out.println(localization.get(LocalizationKey.SELECT_OFFERING_HEADER));
for (int i = 0; i < offerings.size(); i++) {
    CourseOffering o = offerings.get(i);
    String instructor = o.getInstructor() != null ? o.getInstructor().getUsername() : "-";
    String semester = o.getSemester() != null
        ? o.getSemester().getSeason() + " " + o.getSemester().getYear()
        : "-";
    System.out.println((i + 1) + ". " + o.getCourse().getName()
        + " (" + instructor + ") - " + semester);
}
System.out.print(localization.get(LocalizationKey.MENU_CHOICE));
```

7. Communication & Technical Support Module

The communication subsystem allows employees to exchange messages and teachers to create complaints. The technical support subsystem allows users to create technical requests and track their statuses.

- Direct messaging between employees
- Complaint management with urgency levels
- Technical support request system
- Status management for technical requests

SendMessageCommand

```
User found = userRepo.findAll().stream()
    .filter(u -> u.getUsername().equals(receiverUsername))
    .findFirst()
    .orElse(other: null);

if (found == null || !(found instanceof Employee receiver)) {
    System.out.println(localization.get(LocalizationKey.RECEIVER_NOT_FOUND));
    return;
}

if (receiver.equals(sender)) {
    System.out.println(localization.get(LocalizationKey.RECEIVER_NOT_FOUND));
    return;
}

communicationService.sendMessage(sender, receiver, content);
System.out.println(localization.get(LocalizationKey.MESSAGE_SENT_SUCCESS));
```

SendTechRequestCommand

```
User currentUser = userSession.getEffectiveUser();
if (currentUser == null) {
    System.out.println(localization.get(LocalizationKey.ACCESS_DENIED));
    return;
}

System.out.print(localization.get(LocalizationKey.ENTER_REQUEST_DESCRIPTION));
String description = scanner.nextLine().trim();

techSupportService.createRequest(currentUser, description);
System.out.println(localization.get(LocalizationKey.REQUEST_SENT));
```

8. Research Module

The research subsystem allows university members to become researchers, publish papers, subscribe to journals, create research projects, and generate citations.

- Publishing research papers
- Generating citations
- Journal subscriptions
- Research project creation
- Joining research projects
- Top researchers calculation using h-index

CreateResearchProjectCommand

```
System.out.print(s: "Enter research project topic: ");
String topic = scanner.nextLine();

ResearchProject project = new ResearchProject(topic);

researchProjectRepository.addProject(project);

System.out.println(x: "Research project created successfully.");
```

JoinResearchProjectCommand

```
User currentUser = userSession.getCurrentUser();

if (currentUser == null) {
    System.out.println(x: "You must log in first.");
    return;
}

try {
    if (!currentUser.isResearcher()) {
        throw new NotResearcherException(message: "Only researchers can join research projects.");
    }
} catch (NotResearcherException e) {
    System.out.println(e.getMessage());
    return;
}
```

```

List<ResearchProject> projects = researchProjectRepository.findAll();
if (projects.isEmpty()) {
    System.out.println(x: "No research projects available.");
    return;
}

System.out.println(x: "Select research project:");
for (int i = 0; i < projects.size(); i++) {
    System.out.println((i + 1) + ". " + projects.get(i).getTopic());
}
System.out.print(s: "Choice: ");

int choice;
try {
    choice = Integer.parseInt(scanner.nextLine());
} catch (NumberFormatException e) {
    System.out.println(x: "Please enter a valid number.");
    return;
}
if (choice < 1 || choice > projects.size()) {
    System.out.println(x: "Invalid choice.");
    return;
}
ResearchProject selectedProject = projects.get(choice - 1);
selectedProject.addParticipant(currentUser.getId());
researchProjectRepository.saveAll(projects);
System.out.println(x: "You joined the research project successfully.");

```

9. Design Patterns Used

- **Command Pattern:** Every menu action is represented as a separate command class such as PutMarkCommand or JoinResearchProjectCommand.
- **Repository Pattern:** Repositories isolate JSON persistence from business logic.
- **Singleton Pattern:** UserSession guarantees a single active session instance.
- **Decorator Pattern:** ResearcherDecorator dynamically adds research functionality to users.

UserSession

```

public static UserSession getInstance() {
    if (instance == null) {
        instance = new UserSession();
    }
    return instance;
}

```

ResearcherDecorator

```

package unicrm.domain;

public class ResearcherDecorator extends User {

    private final User baseUser;

    public ResearcherDecorator(User baseUser) {
        this.baseUser = baseUser;
        setId(baseUser.getId());
        setUsername(baseUser.getUsername());
        setPassword(baseUser.getPassword());
        setFirstName(baseUser.getFirstName());
        setLastName(baseUser.getLastName());
        setPreferredLanguage(baseUser.getPreferredLanguage());
        setResearcher(researcher: true);
        setJournalSubscriptions(baseUser.getJournalSubscriptions());
    }

    public User getBaseUser() {
        return baseUser;
    }

    @Override
    public String toString() {
        return "Researcher{baseUser=" + baseUser + "}";
    }
}

```

10. JSON Persistence

The project stores runtime data using JSON files instead of a relational database. This approach simplifies development and makes system data easier to inspect and debug.

- users.json
- courses.json
- course_offerings.json
- enrollments.json
- journals.json
- research_projects.json

11. Project Management

The team coordinated development using GitHub and messenger communication. Tasks were divided between team members including UML modeling, backend implementation, testing, and documentation preparation.

Git was used for version control. Team members regularly committed and synchronized changes using pull/fetch operations.

12. Problems and Challenges

- Synchronization between UML diagrams and backend implementation
- Working with JSON serialization using Gson
- Handling console input validation and exceptions

- Encoding problems with Russian/Kazakh text in terminal
- Implementing layered architecture consistently across modules

13. Conclusion

The UniCRM project demonstrates practical usage of Object-Oriented Programming concepts, UML design, design patterns, layered architecture, JSON persistence, and command-based interaction. The system successfully combines academic management, communication, technical support, and research functionality into a single console application.